Digital Implementation of a Signal Conditioning Stage on FPGA for Pulse Simulation in Nuclear Instrumentation

1st Fábio C. Luna
Dept. Elect. Eng.
Federal University of Juiz de Fora
Juiz de Fora, Brazil
https://orcid.org/0009-0002-3133-1729

2nd Thiago C. A. Paschoalin
Dept. Electroeletronic
Fed. Cent. for Technol. Educ. of Minas Gerais
Leopoldina, Brazil
https://orcid.org/0009-0008-4581-5285

3rd Tiago M. Quirino
Dept. Elect. Eng.
Rio de Janeiro State University
Rio de Janeiro, Brazil
https://orcid.org/0000-0002-1799-5222

4th Luciano M. de Andrade Filho
Dept. Elect. Eng.
Federal University of Juiz de Fora
Juiz de Fora, Brazil
https://orcid.org/0000-0003-1792-6793

***Abstract*—In nuclear instrumentation, accurate pulse processing is essential for extracting meaningful information from detector signals. However, developing and validating processing techniques directly on experimental setups is often impractical due to high costs, restricted access, and radiation exposure constraints. Real-time pulse simulators provide a controlled alternative for evaluating signal reconstruction algorithms. A critical component of such simulators is the signal conditioning stage (shaper), whose long-tail impulse response must be faithfully reproduced to capture pile-up and baseline shift effects. While FIR filters are commonly used for digital pulse shaping, they cannot efficiently represent slowly decaying responses, requiring an impractical number of coefficients. This work proposes an IIR-based digital shaper implemented on FPGA, obtained by discretizing the analog transfer function using the Impulse Invariant Method and decomposing it into parallel first- and second-order sections described with fixed-point arithmetic. Validation against an ideal reference confirmed minimal quantization error, and the implementation successfully reproduced the long-tail behavior and baseline shift characteristic of the analog circuit. The resource utilization remained below one percent on the target device, indicating that the proposed shaper can be integrated with other signal reconstruction modules without compromising the FPGA capacity available for additional processing stages.**

***Index Terms*—IIR filter, FPGA, shaper, signal conditioning, nuclear instrumentation, fixed-point arithmetic**

I. Introduction

Nuclear instrumentation is essential for the detection, measurement, and characterization of particles in nuclear and high-energy physics experiments. In this context, different electronic systems are employed to convert microscopic physical phenomena into measurable electrical signals, which can subsequently be acquired, processed, and analyzed [1], [2].

Among these systems, calorimeters are widely used for particle energy measurement. In general, their readout chain consists of multiple stages of signal conditioning and processing. Initially, the interaction of a particle with the sensitive material of the detector produces either electric charge or light, which is then converted into an electrical signal by appropriate devices. This signal is subsequently subjected to an analog shaping stage, whose purpose is to improve the signal-to-noise ratio and to shape the pulse in accordance with the requirements of the digitization stage [2], [3]. Finally, signal processing techniques are applied to reconstruct the deposited energy and to extract relevant parameters from the detected event [1].

Accurate pulse processing is therefore critical for extracting meaningful information from detector signals. However, developing and validating such techniques directly on experimental setups is often impractical due to high costs, restricted access to facilities, and radiation exposure constraints [4], [5]. For this reason, real-time pulse simulators capable of reproducing realistic signal conditions are of great importance, as they provide a controlled environment for the investigation and evaluation of signal reconstruction algorithms and other online processing techniques.

To ensure a more faithful representation of real operating conditions, it is essential to incorporate variability into the generated stimuli, which motivates the use of random or pseudo-random number generators within the simulator [6], [7].

The signal conditioning stage is a key component of both the readout chain and its representation within the simulator, as it adapts the pulse to the subsequent acquisition and processing stages [8], [9]. This stage can be modeled as the combination of two main blocks: the charge-sensitive preamplifier (CSP) and the pulse shaper circuit (PSC), which together determine the overall pulse shape and bandwidth of the system [10]. In addition to enabling proper digitization, this combined response improves the signal-to-noise ratio, limits the bandwidth, and produces a waveform more suitable for the extraction of relevant parameters, such as amplitude and time of occurrence [8], [11], [12].

However, the temporal response associated with this type of shaping may introduce an undesirable long-tail effect in the signal, characterized by a slowly decaying exponential behavior arising from the intrinsic dynamics of the conditioning circuit and the effects of capacitive coupling present in the readout chain. Under high event rate conditions, this long tail increases the likelihood of pile-up and baseline shifts, thereby compromising the accurate estimation of subsequent pulse amplitudes [10]. Typically, digital shapers are implemented using FIR filters, which are well suited for representing finite-duration pulse shapes such as trapezoidal or cusp-like waveforms, as demonstrated by Jordanov and Knoll [13] and Regadío *et al.* [14].

Accurately representing this slowly decaying behavior with a FIR filter would require approximately 10,000 coefficients, resulting in prohibitive hardware resource consumption. In contrast, IIR filters can capture this type of dynamics far more efficiently by exploiting their recursive nature, allowing dominant poles to be represented with a significantly lower implementation order [15].

Motivated by this characteristic, this work proposes the implementation of a digital shaper based on an IIR filter architecture on FPGA, enabling a more realistic simulation of long-tail pulses under pile-up conditions. The filter design, discretization, and fixed-point implementation are described, along with validation results comparing the FPGA output against the ideal response.

II. Implementation

As introduced in Section I, the shaper comprises two cascaded stages: the CSP (Fig. 1) and the PSC (Fig. 2), both extracted from Quirino *et al.* [10]. From a system-level perspective, the shaper is described by an equivalent transfer function that incorporates the combined effects of these two stages.

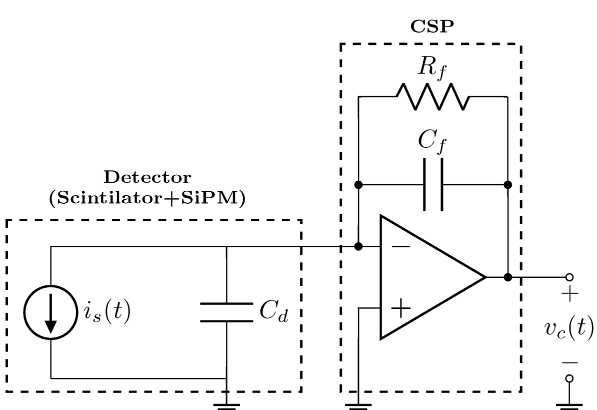

Fig. 1. Charge-sensitive preamplifier (CSP) circuit [10].

Since the two blocks are connected in cascade, the overall system transfer function is obtained as the product of their individual transfer functions:

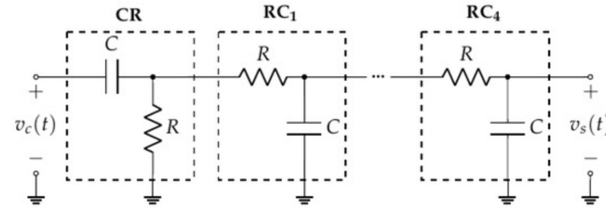

Fig. 2. Pulse shaper circuit (PSC) based on CR-4RC filter topology [10]

$$H(s) = H_{\text{CSP}}(s) \cdot H_{\text{PSC}}(s) \tag{1}$$

The CSP and PSC transfer functions are given by [10]:

$$H_{\text{CSP}}(s) = -\frac{R_f C_d\, s}{R_f C_f\, s + 1} \tag{2}$$

$$H_{\text{PSC}}(s) = \frac{CR\, s}{\begin{gathered}(CR)^5 s^5 + 9(CR)^4 s^4 + 28(CR)^3 s^3 \\ + 35(CR)^2 s^2 + 15CR\, s + 1\end{gathered}} \tag{3}$$

where R_f, C_f, and C_d are the feedback and detector capacitances of the CSP, and R, C are the component values of the CR-4RC stages. The component values were chosen to match the readout system described by Quirino *et al.* [10], with a CSP time constant of $\tau_f = 510$ ns, a shaper time constant of $\tau = 5\,\mu$s, and a sampling period of $T_s = 25$ ns.

The resulting transfer function was then decomposed in terms of its dominant poles, allowing the system dynamics to be represented as a sum of first- and second-order terms:

$$\begin{aligned} H(s) = {} & \frac{-2.28 \times 10^{-4}}{s + 1.00 \times 10^{9}} + \frac{3.23 \times 10^{-3}}{s + 7.10 \times 10^{8}} + \frac{-1.09 \times 10^{-2}}{s + 4.70 \times 10^{8}} \\ & + \frac{3.42 \times 10^{-2}\, s + 7.33 \times 10^{6}}{s^2 + 4.00 \times 10^{8} s + 4.00 \times 10^{16}} + \frac{-6.62 \times 10^{-1}}{s + 2.40 \times 10^{7}} \\ & + \frac{6.36 \times 10^{-1}}{s + 2.00 \times 10^{7}} + \frac{-8.70 \times 10^{-6}}{s + 2.00 \times 10^{3}} \end{aligned} \tag{4}$$

This representation highlights the presence of multiple poles distributed across different time scales, reflecting both the fast response associated with the detector and the preamplifier, and the slower effects introduced by the shaping circuit. In particular, the low-frequency poles are directly related to the presence of long tails in the signal, a typical characteristic of readout systems with capacitive coupling and high-pass stages, which can lead to baseline shift and pile-up effects at high event rates [10].

For its implementation in a digital system, as proposed in this work, it is necessary to obtain an equivalent representation in the discrete-time domain. To this end, the Impulse Invariant Method was adopted. This method is widely used for the discretization of continuous-time systems, as it preserves the impulse response of the original system in the discrete domain [16], [17]. Such a property is particularly important in applications where the temporal characteristics of the signal must be accurately maintained, such as in detector signal processing implemented on FPGA [10].

The discretization is performed based on the following expression:

$$H(z) = \sum_{k=1}^{N} \frac{A_k \left(1 - e^{p_k T_s}\right)}{1 - e^{p_k T_s} z^{-1}} \tag{5}$$

where T_s is the sampling period ($T_s = 25\ ns$) and p_k are the poles of the continuous-time transfer function.

As a result, the discrete-time transfer function can be expressed as:

$$\begin{aligned} H(z) = &\frac{-2.50 \times 10^{-4}}{1 - 1.00 \times 10^{0} z^{-1}} + \frac{2.69 \times 10^{1}}{1 - 6.07 \times 10^{-1} z^{-1}} \\ &+ \frac{-3.03 \times 10^{1}}{1 - 5.49 \times 10^{-1} z^{-1}} \\ &+ \frac{-9.40 \times 10^{0} - 2.70 \times 10^{-2} z^{-1}}{1 + 2.57 \times 10^{-2} z^{-1} + 4.00 \times 10^{-4} z^{-2}} \\ &+ \frac{8.96 \times 10^{0}}{1 - 6.70 \times 10^{-3} z^{-1}} + \frac{4.75 \times 10^{0}}{1 - 6.70 \times 10^{-3} z^{-1}} \\ &- 9.49 \times 10^{-1} \end{aligned} \tag{6}$$

Figure 3 presents the comparison between the normalized continuous-time impulse response of the analog shaper, obtained from Equation (4), and the corresponding discrete-time representation given by Equation (6). The sampled values closely follow the continuous response, confirming that the Impulse Invariant Method preserves the temporal characteristics of the original system after discretization. This agreement validates the discrete-time transfer function as a suitable representation for the subsequent FPGA implementation.

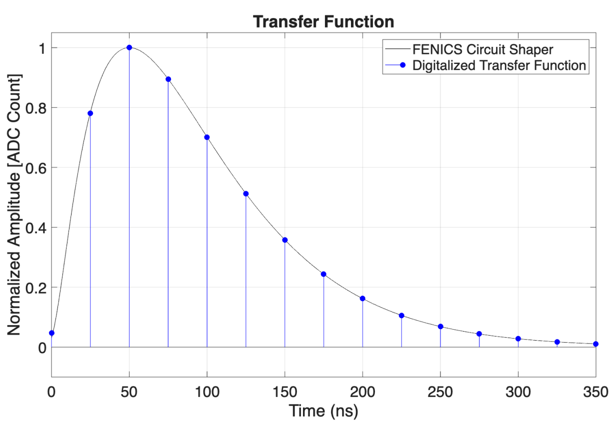

Fig. 3. Comparison between the continuous-time impulse response (FENICS circuit shaper) and the digitalized transfer function obtained via the Impulse Invariant Method.

The difference in the number of components between the continuous- and discrete-time transfer functions can be attributed to pole-zero cancellation or numerical simplifications during discretization. In such cases, poles may be effectively canceled or absorbed, leading to a reduced-order discrete-time representation while preserving the overall system behavior [18], [19]. In particular, some of the resulting sections exhibit denominator coefficients a_k that are numerically negligible, causing the corresponding denominators to collapse to unity and effectively reducing those sections to a constant gain. These contributions are absorbed into a single independent term (-9.49×10^{-1}) in the discrete-time transfer function, which is added directly to the output of the remaining IIR sections.

The discrete-time transfer function $H(z)$ was implemented in hardware targeting an FPGA-based architecture. An Infinite Impulse Response (IIR) structure was adopted due to its recursive nature, enabling an efficient representation of the system with a reduced number of coefficients. A Finite Impulse Response (FIR) approach was not considered, as accurately reproducing the long-tail behavior of the impulse response would require approximately 10,000 coefficients, making its implementation impractical for real-time FPGA-based systems.

The filter was implemented by decomposing $H(z)$ into a parallel arrangement of independent sections, as illustrated in Fig. 4. Specifically, the architecture consists of five first-order sections and one second-order section, each described as an independent digital block, along with a constant term. All seven sections receive the same input sample $x[n]$ simultaneously through a common input bus, and their individual outputs are combined by a summation block, producing the final output $y[n]$ according to:

$$y[n] = \sum_{k=1}^{7} y_k[n] \tag{7}$$

The input-output relationships for each first- and second-order section are given by:

$$y_k[n] = b_{0,k}\, x[n] - a_{1,k}\, y_k[n-1] \tag{8}$$

$$y_k[n] = b_{0,k}\, x[n] + b_{1,k}\, x[n-1] - a_{1,k}\, y_k[n-1] - a_{2,k}\, y_k[n-2] \tag{9}$$

where $b_{i,k}$ and $a_{i,k}$ are the quantized coefficients of the k-th section.

This parallel topology allows all sections to operate simultaneously on the same input sample, maximizing throughput and enabling efficient utilization of the FPGA computational resources.

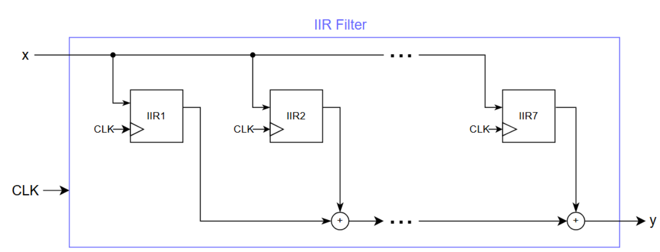

Fig. 4. Block diagram of the IIR filter implementation in FPGA.

As the coefficients are originally represented in floating-point format and FPGAs typically operate using fixed-point arithmetic, a quantization step is required. A commonly adopted representation is the QX.Y format [20], where X denotes the number of integer bits and Y the number of fractional bits. In this work, the input signal was represented in Q1.32 format, and the output in Q1.48 format. This finite word-length representation inherently introduces quantization errors, whose impact on the output signal fidelity is evaluated in the Results section.

Figure 5 illustrates the internal architecture of a first-order IIR section. Since the coefficients are scaled by 2^{15}, this gain must be removed from the output before it is fed back through the delay element z^{-1}, preventing overflow and ensuring numerical stability in the recursive computation. The same principle applies to the second-order section.

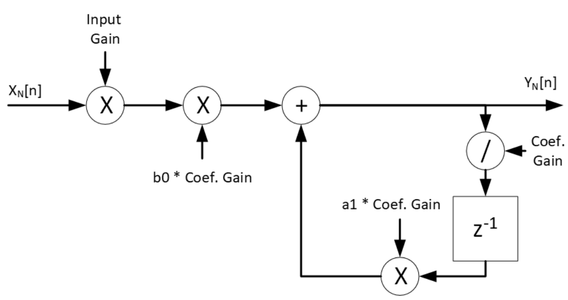

Fig. 5. Internal architecture of a first-order IIR section with gain compensation in the feedback path.

III. Results

To evaluate the proposed IIR-based shaper implementation, two aspects were analyzed: the fidelity of the temporal response compared to the reference model, and the hardware resource utilization on the target FPGA.

A. Temporal Response Fidelity

The impulse response of the FPGA implementation was compared against the reference model. Figure 6 presents the superimposed responses obtained from both platforms. The results show that the discrete-time IIR implementation closely reproduces the expected waveform, including the main pulse shape and the long-tail decay characteristic of the shaping circuit.

To quantify the agreement between the two responses, the root mean square error (RMSE) between the FPGA output and the ideal signal was computed as:

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{n=0}^{N-1}\left(y_{\text{FPGA}}[n] - y_{\text{ideal}}[n]\right)^2} \tag{10}$$

The obtained RMSE was 1.13×10^{-4}, confirming that the fixed-point quantization introduced minimal degradation in the signal representation. Figure 7 presents the sample-by-sample error between the ideal response and the FPGA output. The error remains bounded within the order of 10^{-3} throughout the observation window, indicating that the chosen fixed-point representation provides sufficient numerical precision to faithfully reproduce the shaper dynamics, including the long-tail region.

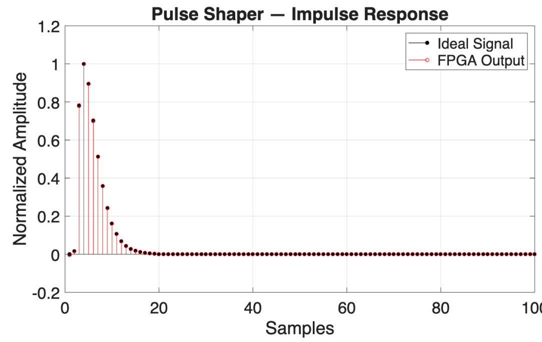

Fig. 6. FPGA output compared to the ideal impulse response

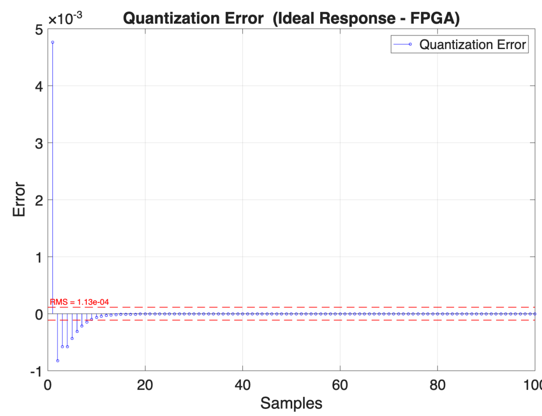

Fig. 7. Quantization error between the ideal response and the FPGA implementation. The dashed line indicates the RMS error level (1.13×10^{-4}).

To complement the error analysis, Figure 8 presents the sample-by-sample percentage difference between the FPGA output and the ideal response, normalized by the peak amplitude of the ideal signal:

$$e_{\%}[n] = \frac{y_{\text{FPGA}}[n] - y_{\text{ideal}}[n]}{\max(|y_{\text{ideal}}|)} \times 100\% \tag{11}$$

The percentage error remains below 0.476% for all samples, further confirming the fidelity of the fixed-point implementation.

Figure 9 presents a detailed view of the long negative tail region of the impulse response. The close overlap between the reference and the FPGA output confirms that the IIR-based implementation accurately preserves the slowly decaying behavior characteristic of the shaper circuit. This result is particularly relevant, as the faithful reproduction of the long-tail dynamics is essential for the correct evaluation of pile-up and baseline shift effects in high event rate scenarios.

Figure 10 illustrates the baseline shift effect by presenting the FPGA shaper output when a continuous stream of pulses is applied, with amplitudes generated using pseudo-random

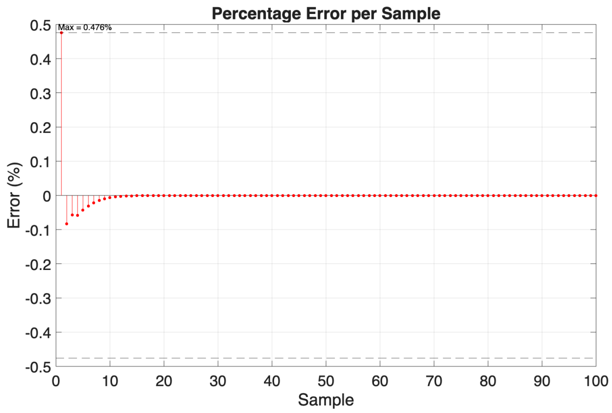

Fig. 8. Sample-by-sample percentage difference between the FPGA output and the ideal response, normalized by the peak amplitude of the reference signal.

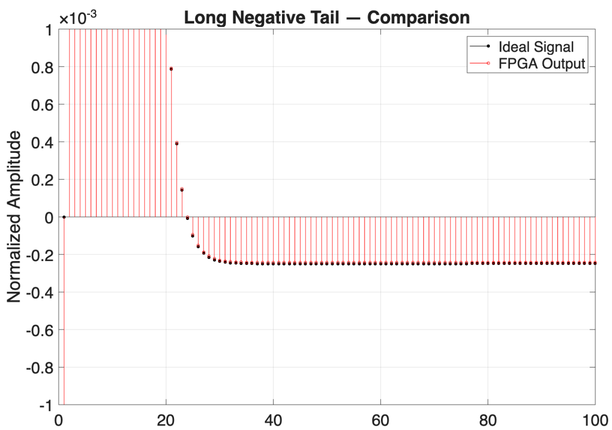

Fig. 9. Comparison between the ideal and FPGA impulse responses in the long negative tail region.

number generators [6], [7]. The progressive deviation of the baseline from zero confirms that the IIR implementation faithfully reproduces the pile-up and baseline drift effects expected from the analog circuit.

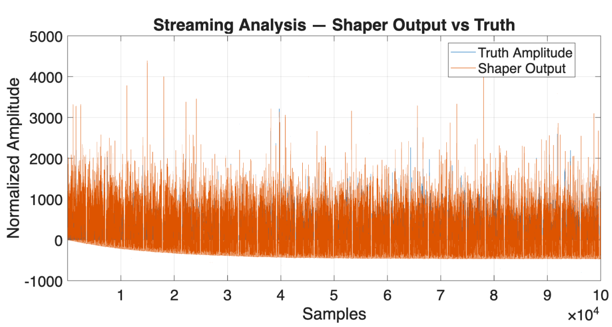

Fig. 10. Streaming analysis of the FPGA shaper output compared to the true pulse amplitudes, showing the baseline shift effect caused by the long-tail response.

A zoomed view of this signal is shown in Figure 11, where the displacement of the shaper output below the true baseline can be more clearly observed across individual pulses.

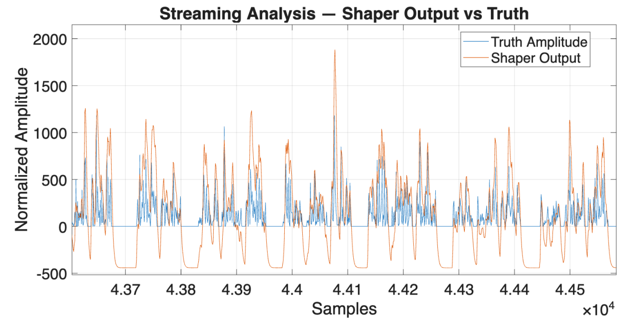

Fig. 11. Zoomed view of the streaming analysis, highlighting the baseline shift between the shaper output and the true pulse amplitudes.

These results demonstrate that the proposed IIR-based architecture is a viable solution for the digital representation of the analog shaping stage, capable of reproducing both the impulse response characteristics and the baseline shift effects observed in real readout systems, without requiring the large number of filter coefficients that a FIR-based approach would demand.

B. *FPGA Resource Utilization*

Table I summarizes the hardware resource utilization of the proposed IIR shaper implementation. The results confirm that the parallel decomposition into first- and second-order sections yields a compact architecture with low resource consumption.

TABLE I
FPGA RESOURCE UTILIZATION

Resource	Used	Available	Utilization (%)
LUTs	2254	242400	0.93
Registers	357	484800	0.07
DSP Blocks	10	1920	0.52
Block RAM	0	600	0.00

The low resource footprint demonstrates the suitability of the IIR-based approach for integration into larger FPGA-based pulse simulation systems. Minimizing the hardware consumption of the shaper is particularly important in this context, as it maximizes the remaining FPGA resources available for the implementation and evaluation of online signal reconstruction techniques and other real-time processing algorithms within the same device.

Table II presents the timing performance of the shaper implementation after place and route. The design was constrained with a target clock frequency of 40 MHz.

TABLE II
TIMING ANALYSIS RESULTS

Parameter	Value
Target Clock Frequency	40 MHz
Clock Period	25.000 ns
Worst Negative Slack (WNS)	+18.375 ns
Maximum Operating Frequency (F_{max})	150.9 MHz

The positive WNS of +18.375 ns confirms that all timing constraints are met. The target clock of 40 MHz operates well below the maximum operating frequency of approximately 151 MHz, ensuring reliable operation with significant timing margin.

This timing margin indicates that the proposed shaper can be integrated with the remaining modules of the pulse simulator within the same FPGA device, without compromising the overall timing closure of the system.

IV. Conclusion

This work addressed the digital implementation of a signal conditioning stage (shaper) on FPGA as part of a real-time pulse simulator for nuclear instrumentation. Such simulators are essential for enabling the development and evaluation of signal processing techniques in a controlled environment, avoiding the practical limitations associated with direct experimentation on detection systems.

Although FIR filters are the most common choice for digital pulse shaping, they become impractical when the shaper response exhibits a slowly decaying long tail, as is the case of unipolar shapers operating under pile-up conditions. To overcome this limitation, an IIR-based architecture was proposed, derived from the cascade of a charge-sensitive preamplifier (CSP) and a CR-4RC pulse shaper circuit (PSC), discretized through the Impulse Invariant Method and decomposed into a parallel arrangement of first- and second-order sections.

The FPGA implementation, using fixed-point arithmetic, was validated against an ideal reference, showing minimal degradation due to quantization and faithful reproduction of both the impulse response and the baseline shift effect, which arises from the long tail introduced by the capacitive effects of the conditioning circuit. These results confirm the IIR-based architecture as a viable digital representation of the analog shaping stage, without requiring the prohibitive number of coefficients that a FIR-based approach would demand.

Synthesis and timing analysis on the Xilinx KCU105 board showed that the proposed shaper consumes less than 1% of the available FPGA resources and operates with significant timing margin relative to the target clock frequency. This low footprint, combined with the available timing slack, indicates that the shaper can be integrated with the remaining modules of the pulse simulator, such as random number generators, noise injection blocks, and online signal reconstruction algorithms, within the same device and without compromising the overall system performance.

Future work includes the integration of the proposed shaper with a complete FPGA-based pulse simulation chain and the evaluation of baseline correction and signal reconstruction techniques applied to the shaper output under high pile-up conditions.

Acknowledgment

This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior-Brasil (CAPES)—Finance Code 001. The authors are thankful to CNPq (Conselho Nacional de Desenvolvimento Científico e Tecnológico) and RENAFAE (Rede Nacional de Física de Altas Energias) for their support.

References

[1] R. K. Carnegie *et al.*, "Particle detectors at accelerators," in *Review of Particle Physics*, Prog. Theor. Exp. Phys., 2020.

[2] H. Spieler, *Semiconductor Detector Systems*. Oxford, U.K.: Oxford Univ. Press, 2005.

[3] T. C. A. Paschoalin, T. M. Quirino, J. P. B. S. Duarte, and L. M. de Andrade Filho, "Learned Soft-Thresholding for Streaming Sparse Signal Recovery in Nuclear Instrumentation," *IEEE Transactions on Nuclear Science*, vol. 73, no. 2, pp. 282–290, 2026, doi: 10.1109/TNS.2025.3644239.

[4] D. Ponikvar, "An FPGA-based nuclear pulse generator with a prescribed amplitude distribution," *Nucl. Instrum. Methods Phys. Res. A*, vol. 877, pp. 71–78, 2018.

[5] A. García-Durán, V. M. Hernández-Dávila, H. R. Vega-Carrillo, O. O. Ordaz-García, and others, "Nuclear pulse generator embedded in FPGA," *Appl. Radiat. Isot.*, vol. 147, pp. 129–135, 2019.

[6] T. C. A. Paschoalin, U. R. F. Dias, M. S. Aguiar, D. F. dos Santos, T. M. Quirino, and L. M. de Andrade Filho, "Uncorrelated Pseudo-Random Generator for FPGA," in *2025 38th SBC/SBMicro/IEEE Symposium on Integrated Circuits and Systems Design (SBCCI)*, 2025, pp. 1–5, doi: 10.1109/SBCCI66862.2025.11218676.

[7] T. C. A. Paschoalin, T. M. Quirino, and L. M. de Andrade Filho, "Multi-Memory Approach for Random Number Generators in FPGA," *Applied Sciences*, vol. 16, art. 2537, 2026, doi: 10.3390/app16052537.

[8] I. Hafizh, D. M. Smith, L. Abbene, G. Gerardi, and M. Sampietro, "Assessment of analog pulse processor performance for high-rate X-ray spectroscopy," *Nucl. Instrum. Methods Phys. Res. A*, vol. 945, art. 162479, 2019.

[9] M. Anderson *et al.*, "A readout system for the STAR time projection chamber," *Nucl. Instrum. Methods Phys. Res. A*, vol. 499, no. 2–3, pp. 659–678, 2003.

[10] T. M. Quirino, T. C. A. Paschoalin, G. I. Gonçalves, P. H. B. Lisboa, L. M. de Andrade Filho, and B. S.-M. Peralva, "Online Pulse Compensation for Energy Spectrum Determination: A Pole-Zero Cancellation and Unfolding Approach," *Electronics*, vol. 14, art. 493, 2025, doi: 10.3390/electronics14030493.

[11] A. Regadío, J. Tabero, S. Sánchez-Beato, G. Espinosa, and J. M. Monzó-Cabrera, "Impact of colored noise in pulse amplitude measurements: A time-domain approach using differintegrals," *Nucl. Instrum. Methods Phys. Res. A*, vol. 811, pp. 25–31, 2016.

[12] L. Abbene, G. Gerardi, G. Raso, S. Basile, M. Brai, and F. Principato, "Energy resolution and throughput of a new real time digital pulse processing system for X-ray and gamma ray semiconductor detectors," *J. Instrum.*, vol. 8, no. 7, P07019, 2013.

[13] V. T. Jordanov and G. F. Knoll, "Digital synthesis of pulse shapes in real time for high resolution radiation spectroscopy," *Nucl. Instrum. Methods Phys. Res. A*, vol. 345, no. 2, pp. 337–345, 1994.

[14] A. Regadío, S. Sánchez-Prieto, M. Prieto, and J. Tabero, "Implementation of a real-time adaptive digital shaping for nuclear spectroscopy," *Nucl. Instrum. Methods Phys. Res. A*, vol. 735, pp. 297–303, 2014.

[15] C. Bobin, J. Bouchard, and B. Censier, "Digital pulse processing and optimization of the front-end electronics for nuclear instrumentation," *Appl. Radiat. Isot.*, vol. 87, pp. 200–206, 2014.

[16] RABINER, L. R.; GOLD, B. *Theory and Application of Digital Signal Processing*. Englewood Cliffs: Prentice-Hall, 1975.

[17] T. W. Parks and C. S. Burrus, *Digital Filter Design*, Topics in Digital Signal Processing, Wiley, 1987.

[18] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed., Pearson, 2010.

[19] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed., Prentice Hall, 2007.

[20] PASCHOALIN, T. C. A.; QUIRINO, T. M.; ANDRADE FILHO, L. M. de A. Implementação em FPGA do circuito do condicionamento de sinal do calorímetro hadrônico do ATLAS. In: *Anais do Simpósio Brasileiro de Automação Inteligente / Simpósio Brasileiro de Sistemas Elétricos (SBAI/SBSE)*, 2025.

[21] AMD, "Kintex UltraScale FPGA KCU105 Evaluation Kit," [Online]. Available: https://www.amd.com/en/products/adaptive-socs-and-fpgas/evaluation-boards/kcu105.html. Accessed: Apr. 16, 2026.